You now have a list of your above average drivers by trip count, in descending order based on the trips they've provided.

How can you adapt this Subquery to Active Record?

Taking inspiration from *Advanced Active Record: Using Subqueries in Rails*[13] on the PgAnalyze blog, take the techniques and write or review the Active Record code below. Run this code in bin/rails console and confirm that the generated SQL is the same.

ruby/active_record_subquery.rb

```
Driver.where('trips_count > (:avg)', avg: Driver.select('avg(trips_count)')).
  order(trips_count: :desc).
  limit(5)
```

In this code, :avg is a placeholder and the value is supplied in a Hash key and value style. Instead of SQL query result rows, you're now working with Active Record objects. Pretty cool.

Subquery Expressions in SQL are very useful but they can become difficult to read. Another advanced query design technique called Common Table Expressions (CTEs) helps make multi-query statements better organized and easier to read and maintain.

Using Common Table Expressions (CTE)

Common Table Expressions abbreviated as a CTE,[14] are a technique in PostgreSQL to combine two or more queries into a single statement with named parts.

CTEs are also called WITH queries because they use the WITH keyword. They'll be referred to as CTEs in this chapter.

CTEs combine queries that are independent but can be used to work together to pull data from multiple places. CTEs can be used recursively by combining a non-recursive portion and a recursive portion and then using a UNION to combine them. Recursive CTEs are beyond the scope of this chapter, but the PostgreSQL Tutorial has a nice example showing how to find direct reports for a manager using the technique.[15]

In this section, you'll create a CTE to answer questions about Rideshare Drivers. You'll look at new Drivers on the platform. As you did in the last

13. https://pganalyze.com/blog/active-record-subqueries-rails
14. https://www.postgresql.org/docs/current/queries-with.html
15. https://www.postgresqltutorial.com/postgresql-tutorial/postgresql-recursive-query/